

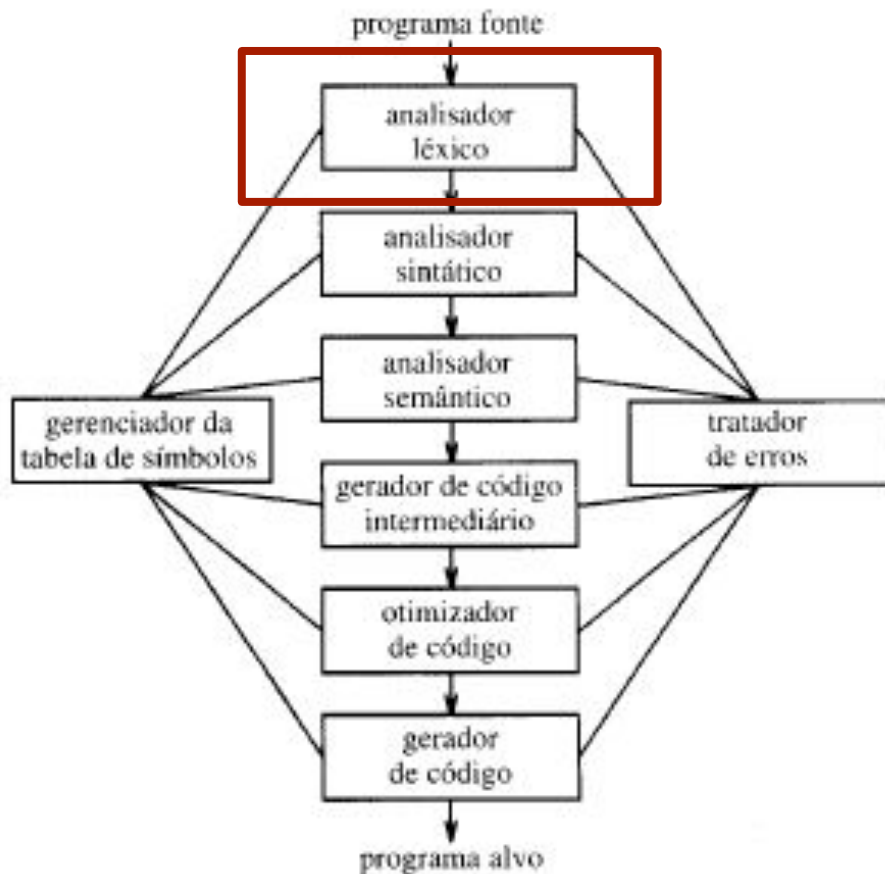
Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 5

Análise Léxica

Introdução



Introdução

- **Análise Léxica**

- Também pode ser chamada de **fase de leitura (scanning)**
- Lê o fluxo de caracteres que compõem o programa fonte e os agrupa em sequências significativas, chamadas **lexemas**
- Para cada lexema, o analisador léxico produz como saída um **átomo**

Introdução

- **Análise Léxica**
 - Que se encarrega de isolar as **palavras-chaves, símbolos especiais, símbolos especiais compostos, identificadores e constantes**, transformando-os em códigos convenientes

Introdução

- **Análise léxica**
 - Faz a interface entre o texto fonte e as demais partes.
 - Partindo do **texto fonte de entrada** (qualquer sequência de caracteres), **identifica os símbolos terminais da gramática** (da linguagem em questão)

Símbolos

- O Pascal é uma GLC, cujo vocabulário são os símbolos:
 - Letras: a,b,c.....,z
 - Dígitos: 0,1,2.....9
 - Símbolos Especiais: . ; , () := < > + - *
 - Símbolos Especiais Compostos: := (* *)
 - Símbolos Compostos em Negrito: **program, label, var, procedure, function, begin, end, if, then, else, while, do, or, and, div, not**

Símbolos

- O Analisador Léxico a ser desenvolvido fará uma verificação em um programa escrito em Pascal

Símbolos

- Para que não haja confusão entre palavras reservadas, identificadores e números, deverá haver espaços em branco separando os mesmos
 - Exemplo
 - **ifa** e **while35** são identificadores
 - **if a** e **while 35** representam palavras reservadas, seguidas de identificadores ou números

Símbolos

- Uma regra (raramente explicitada nos manuais das linguagens) diz que um token se estende até que seja encontrado um caracter que não faz parte dele
- Essa regra, (ou alguma regra semelhante) é necessária para que o analisador léxico possa reconhecer em **XYZ123A+1** uma ocorrência de um identificador **XYZ123A**, e não, por exemplo, ocorrências de três identificadores **XY**, **Z123** e **A**.

Símbolos

- Usando a regra mencionada, é a presença do caracter + que determina o fim do identificador. Essa regra nos obriga a separar palavras reservadas e identificadores por brancos ou outros caracteres.
 - Por exemplo, em

IF XYZ=ABC+DEF THEN XYZ:=ABC DIV DEF;

Símbolos

- Pascal não faz distinção entre maiúsculas e minúsculas em identificadores e palavras reservadas, mas algumas linguagens (como C) tratam dois identificadores como **TabSimb** e **tabsimb** como distintos.

Átomos

- É uma sequência de caracteres que representa um símbolo terminal da linguagem
- **Exemplo em Pascal:**
 - begin, +, :=, indice, 308
- Em geral, a cada símbolo da linguagem associa-se um código distinto

Átomos

- O **analisador léxico** processa o programa-fonte, desenvolvendo os códigos dos **átomos** encontrados
- Em relação aos **átomos** (que são **identificadores ou números**), deve ser desenvolvida também a própria sequência de caracteres, ou outra informação equivalente (como a posição na tabela de símbolos, valor do número em representação interna)

Exemplo

```
    var i, j : integer;  
begin  
  i = 0;  
  j = 0;  
  ...  
end
```

Exemplo

```
var i, j : integer;  
begin  
i = 0;  
j = 0;  
...  
end
```



poderia produzir a seqüência de *tokens*

27	45	60	45	61	14	62
05						
45	63	40	62			
45	63	40	62			
...						
10						

Exemplo

```
var i, j : integer;  
begin  
i = 0;  
j = 0;  
...  
end
```



poderia produzir a seqüência de *tokens*

```
27 45 60 45 61 14 62  
05  
45 63 40 62  
45 63 40 62  
...  
10
```

var	27	identificador	45	,	60
:	61	integer	14	begin	05
=	63	numero	40	;	62
end	10				

Implementação

- O Analisador Léxico a ser desenvolvido fará uma verificação em um programa escrito em Pascal
- Comparada a outras partes do compilador, o **analisador léxico é de fácil implementação**
- **Autômatos finitos**
 - Utiliza-se os autômatos finitos para representar os blocos dentro dos Pascal Simplificado

Implementação

- **Autômatos finitos**

- Muito pouco da teoria dos autômatos finitos é utilizado na implementação
- A sua aplicação baseia-se no fato de que a estrutura dos tokens de uma linguagem é, no geral, muito simples

Implementação

- Os aspectos importantes do Pascal que não serão incluídos na nossa versão são:
 - **Tipos real e char**
 - **Declarações de constantes**
 - **Tipos, registros, apontadores, arquivos**
 - **Comentários**

Implementação

- **Funções do Analisador Léxico**
 - Leitura e impressão do programa-fonte (fins de linha, de arquivo, etc.)
 - Reedição em formato mais legível usando identificadores
 - Extração e classificação de átomos
 - Eliminação de delimitadores e comentários
 - Diagnóstico e tratamento de alguns erros
 - **Conversão numérica**

Conversão numérica

- Sequência que representa número
- Exemplos:

- 10.8

- átomo: 10.8
- símbolo: c_real



O analisador léxico
separa o número

Conversão numérica

- Sequência que representa número
- Exemplos:
 - 1
 - átomo: 1
 - símbolo: c_inteiro

Conversão numérica

- Sequência que representa número
- Exemplos:
 - 0
 - átomo: 0
 - símbolo: `c_inteiro`

Conversão numérica

- Sequência que representa número
- Exemplos:
 - ponto (.)
 - átomo: .
 - símbolo: c_ponto

Conversão numérica

- Sequência que representa número
- Exemplos:
 - 10.8
 - átomo: 10.8
 - símbolo: c_real

Algoritmo

- Palavras em letra maiúsculas
 - funções (PROXIMO, CODIGO, ERRO)
- Palavras em letras minúsculas em negrito
 - palavras reservadas (**procedimento, inicio, fim, etc.**)
- Palavras em letras minúsculas
 - demais identificadores (**proximo, simbolos_especiais**)
- Para implementar esse algoritmo é necessário desenvolver os procedimentos: **PROXIMO, CODIGO, ERRO**

Algoritmo

- Dentro de **CODIGO** acontecerá a classificação dos lexemas em:
 - **codigo_de_identificador**
 - **codigo_de_numero**
 - Ou ele recebe um valor já determinado, por ser uma **palavra reservada ou símbolo**
- Devolve na variável global **simbolo** o código do átomo encontrado

Algoritmo

- Dentro do procedimento **PROXIMO**:
 - Coloca na variável **proximo** o próximo caractere da cadeia de entrada
 - Quando for “□”, deve ler a próxima linha (registro)
 - É necessário verificar o fim do arquivo
 - Também é preciso um contador para informar a linha do erro (dentro de **PROXIMO**)

Algoritmo

- Dentro de **ERRO** será mostrado uma mensagem de erro ao usuário e a linha do erro
- **Não são muitos os erros que são detectados** pelo analisador léxico
- **Exemplos:**
 - Símbolo especial desconhecido
 - Identificador ou número mal formados

Algoritmo

procedimento ANALISADOR_LEXICO;
inicio

átomo := cadeia_vazia;

enquanto próximo = '□' **faça** PROXIMO;

se próximo $\in$ símbolos_especiais **então**

inicio

s := próximo; PROXIMO;

caso s **seja**

'.' : **se** próximo = '=' **então** s := ':='; PRÓXIMO;

outros: nada

fim do caso; simbolo := CODIGO (s);

fim se

**Representa o fim da
linha**



Algoritmo

```
senão  
início  
  se proximo ∈ letras então  
    início  
      repita  
        atomo := atomo & proximo ; PRÓXIMO;  
      até proximo ∉ letras_e_digitos;  
    se atomo ∈ palavras_reservada então  
      símbolo := CÓDIGO(átomo);  
    senão  
      símbolo := codigo_de_identificador;  
  fim se  
fim se
```



Concatenação

Algoritmo

senão

se proximo $\in$ dígitos **então**

início

repita

átomo := átomo & próximo ;

PRÓXIMO;

até próximo $\notin$ dígitos

se próximo $\in$ letras **então** ERRO;

símbolo := código_de_numero;

fim se

senão ERRO;

fim

Tarefas adicionais do Analisador Léxico

- Converter todas as letras para maiúsculas (no caso do Pascal) para economia de memória
- Eliminar todos os caracteres desnecessários
- Numerar o arquivo para melhor procura dos erros
- Analisar todos os caracteres especiais
- Separar todas as palavras reservadas em símbolos